

StockEasy

Regras de Negócio e Catálogo de Erros

2026-05 · DESAFIO TÉCNICO

Regras de Negócio

Documento de referência para o avaliador: entidades canônicas, enums, regras enforced pelo backend e o catálogo fechado de `errorCodes` que vocabulariza toda resposta de erro da API. As regras aqui são as que viram **invariantes de teste** no `Inventory.Tests/` (Plano 04-01/02/03).

Entidades

Product

Mapeada para a tabela `products` (`init.sql`). POCO com setters mutáveis (Dapper materializa).

Campo	Tipo C#	Coluna SQL	Constraint	Descrição
Id	Guid	id uuid PK	DEFAULT gen_random_uuid()	Server-generated UUID v4.
Code	string	code varchar(50)	NOT NULL UNIQUE (sem filtro de deleted_at)	Código único do produto; anti-reuso por design.
Description	string	description varchar(200)	NOT NULL	Descrição livre do produto.
Type	ProductType	type int	CHECK type IN (0,1,2)	Enum (Electronic / Appliance / Furniture).
SupplierValue	decimal	supplier_value numeric(18,2)	≥ 0	Custo por unidade pago ao fornecedor (BRL).
StockQuantity	int	stock_quantity int	≥ 0	Saldo on-hand.
DeletedAt	DateTime?	deleted_at timestampz NULL	NULL = ativo	Soft delete timestamp (UTC).
CreatedAt	DateTime	created_at timestampz	DEFAULT now()	UTC.

Campo	Tipo C#	Coluna SQL	Constraint	Descrição
UpdatedAt	DateTime	updated_at timestampz	DEFAULT now()	UTC, atualiza em cada mutação.

StockMovement

Maapeada para `stock_movements`. Imutável após o INSERT (MOVE-11).

Campo	Tipo C#	Coluna SQL	Constraint	Descrição
Id	Guid	id uuid PK	DEFAULT gen_random_uuid()	Server-generated UUID v4.
ProductId	Guid	product_id uuid	FK → products(id)	FK ao produto.
Type	MovementType	type int	CHECK type IN (0,1)	Enum (Inbound / Outbound).
Quantity	int	quantity int	CHECK quantity > 0	Sempre positiva (direção codificada em <code>type</code>).
SupplierValue	decimal?	supplier_value numeric(18,2) NULL	Inbound: NOT NULL · Outbound: NULL	Custo por unidade pago no Inbound.
SaleValue	decimal?	sale_value numeric(18,2) NULL	Outbound: NOT NULL · Inbound: NULL	Preço por unidade vendido no Outbound.
IdempotencyKey	Guid	idempotency_key uuid	NOT NULL UNIQUE	Chave de idempotência (UUID v4 cliente).
OccurredAt	DateTime	occurred_at timestampz	DEFAULT now()	Business-time UTC.
CreatedAt	DateTime	created_at timestampz	DEFAULT now()	Insertion UTC.

Enums

ProductType

Catálogo fechado de categorias de produto. Persistido como `int` em `products.type` ; serializado como string no JSON via `JsonStringEnumConverter` global.

Identifier (EN)	Ordinal	Rótulo PT-BR
Electronic	0	Eletrônico
Appliance	1	Eletrodoméstico
Furniture	2	Móvel

Os rótulos PT-BR ficam em `frontend/src/shared/labels.ts` (`productTypeLabel`) — centralizados conforme UX-05.

MovementType

Direção do movimento de estoque.

Identifier (EN)	Ordinal	Rótulo PT-BR
Inbound	0	Entrada
Outbound	1	Saída

Persistido como `int` em `stock_movements.type` ; serializado como `"Inbound"` / `"Outbound"` no JSON (AGENT-01).

Regras Enforced

Lista canônica de invariantes que o backend enforça — cada bullet mapeia ao código que enforce + ao requisito que pede.

- **Soft delete em produtos** — `DELETE /api/products/{id}` faz `UPDATE products SET deleted_at = now() WHERE id = @id AND deleted_at IS NULL` . Listagens filtram `WHERE deleted_at IS NULL` por padrão; `?includeDeleted=true` retorna todos. Histórico de movimentos do produto deletado **permanece acessível** (auditoria preservada). Enforce em `ProductService.DeleteAsync` (PROD-05, MOVE-11).
- **Code é único e NÃO reutilizável** — `init.sql` define `products.code` `UNIQUE` **sem filtro** de `deleted_at` . Re-cadastro com mesmo código de produto soft-deleted retorna `DUPLICATE_CODE` . Proteção anti-reuso é uma propriedade do schema (PROD-06 enforcement gratuito no nível do banco).

- **Movimentos são imutáveis** — não existem endpoints `PUT`, `DELETE` ou `PATCH` em `/api/stock-movements`. Tentativa retorna `405 Method Not Allowed` (route absence). Histórico é auditoria — não se edita o passado (MOVE-11).
- **Idempotency obrigatória em movimentos** — header `Idempotency-Key` (UUID v4) é **required** em `POST /api/stock-movements`. Ausente → `400 MISSING_IDEMPOTENCY_KEY` com hint instruindo `crypto.randomUUID()`. Replay com mesma chave → `200 OK` (não `201`) + header `Idempotency-Replay: true` + body byte-idêntico ao da primeira resposta. Implementado em `StockMovementService.CreateAsync` (MOVE-02, MOVE-03).
- **Read-modify-write com SELECT FOR UPDATE** — em `POST /api/stock-movements`, o service abre transação, faz `SELECT ... FOR UPDATE` no produto (BACK-10), valida saldo e invariants, INSERT do movimento, UPDATE do produto e COMMIT — atômico. Sem race conditions em concorrência (MOVE-01).
- **Race-window recovery via 23505** — o INSERT do movimento captura `PostgresException(SqlState='23505')` (race em `idempotency_key` UNIQUE), faz rollback, re-fetch pela chave e retorna replay. Sem efeito duplicado mesmo sob retransmissão concorrente (MOVE-03).
- **Saldo insuficiente bloqueado** — `Outbound` com `quantity > stockQuantity` → `422 INSUFFICIENT_BALANCE` com hint dinâmica ("Reduza a quantidade para no máximo {available} ou registre uma entrada antes.") e `details.deficit`. Enforce em `StockMovementService.CreateAsync` antes do INSERT (MOVE-04).
- **Produto deletado não aceita movimento** — `POST /api/stock-movements` em produto com `deleted_at != NULL` → `422 PRODUCT_DELETED` com hint mencionando o `productCode` (MOVE-05).
- **Value-field invariant Inbound/Outbound** — `Inbound` requer `supplierValue` e **proíbe** `saleValue`; `Outbound` é o espelho (requer `saleValue`, proíbe `supplierValue`). Inversão → `422 INVALID_MOVEMENT_VALUES` com hint dinâmica citando `expectedField` e `providedField` (MOVE-07).
- **Zero N+1 em histórico** — `GET /api/stock-movements` faz **um JOIN'd SELECT** trazendo `productCode` + `productDescription` + um COUNT separado. Exatamente **2 SQL statements** por página, independente do `pageSize`. Verificado em produção via `log_statement='all'` do Postgres (evidência em `.planning/phases/03-stock-movements-vertical-slice/03-VERIFICATION.md` §MOVE-09). Enforce em `StockMovementRepository.ListAsync` (MOVE-09, BACK-11).
- **Paginação obrigatória** — `?page=1&pageSize=30` em toda listagem; clamp `pageSize ∈ [1, 100]` validado por `PaginationValidator`; fora dos limites → `400 VALIDATION_ERROR` (BACK-15).
- **Enums serializados como string** — `JsonStringEnumConverter` global em `Program.cs`. Respostas trazem `"type": "Inbound"`, não `"type": 0`. LLM e humano leem melhor (AGENT-01).

Catálogo de errorCodes

Vocabulário **fechado** em SCREAMING_SNAKE_CASE. Adicionar um novo **errorCode** requer atualização desta tabela. Cada slug é construído no constructor de uma **DomainException** tipada (em **backend/Inventory/Exceptions/**) com **hint** dinâmica usando dados reais do contexto (AGENT-08, D20).

errorCode	HTTP	Category	Exception	Hint (PT) dinâmica
VALIDATION_ERROR	400	VALIDATION	FluentValidation.ValidationException (via middleware)	"Corrija listados 'details.' novame
MISSING_IDEMPOTENCY_KEY	400	VALIDATION	MissingIdempotencyKeyException	"Gere u cliente (crypto. e envie 'Idempo
PRODUCT_NOT_FOUND	404	NOT_FOUND	ProductNotFoundException	"Produ encontr lista e t novame
MOVEMENT_NOT_FOUND	404	NOT_FOUND	MovementNotFoundException	"Movim encontr página novame
DUPLICATE_CODE	422	BUSINESS_RULE	DuplicateCodeException	"Já exis com có Use out recuper via /api/ includel
PRODUCT_DELETED	422	BUSINESS_RULE	ProductDeletedException	"Produ excluído não po registra produto
INSUFFICIENT_BALANCE	422	BUSINESS_RULE	InsufficientBalanceException	"Reduza para no {availab uma ent

errorCode	HTTP	Category	Exception	Hint (PT dinâmico)
INVALID_MOVEMENT_VALUES	422	BUSINESS_RULE	InvalidMovementValuesException	"Movim '{type}' campo '{expect não '{pr
INTERNAL_ERROR	500	INTERNAL	(qualquer Exception não tratada — fallback do middleware)	"Tente r alguns i persistir suporte traceld.

Exemplo de ErrorResponse (canonical)

Toda resposta de erro segue o envelope canônico (AGENT-06). Exemplo real capturado em produção contra o produto de smoke `P3-SMOKE-001` (stock 55), tentando `Outbound` com `quantity=999999` (evidência em `.planning/phases/03-stock-movements-vertical-slice/03-VERIFICATION.md §MOVE-04`):

```
{
  "errorCode": "INSUFFICIENT_BALANCE",
  "category": "BUSINESS_RULE",
  "message": "Saldo insuficiente: solicitado 999999 unidades, disponível 55",
  "hint": "Reduza a quantidade para no máximo 55 ou registre uma entrada antes.",
  "statusCode": 422,
  "retryable": false,
  "details": {
    "productId": "7168c579-8259-4c0e-949b-998a7147c47f",
    "productCode": "P3-SMOKE-001",
    "requested": 999999,
    "available": 55,
    "deficit": 999944
  },
  "traceId": "0HNLJG2QH4IQ8:00000001",
  "timestamp": "2026-05-16T19:26:26.1301069Z"
}
```

Esta é a forma agentic-friendly do envelope: `errorCode` (slug estável que o LLM aprende uma vez), `category` (roteamento), `message` (texto humano em PT), `hint` (instrução acionável com **dados reais**), `statusCode` duplicado no body (LLM não precisa parsear o status HTTP), `retryable` (decisão automática), `details` (objeto estruturado com toda a informação que poderia ser usada para recovery), `traceId` (correlação com logs) e `timestamp` (ISO 8601 UTC).

Idempotência — Fluxo Completo

`POST /api/stock-movements` é a única mutação sensível do MVP (cadastro de produto tem idempotência natural pelo `UNIQUE` em `code`). O fluxo de idempotência tem **duas paths que produzem o mesmo output** — referência cruzada com o Diagrama 3 em `02-architecture.md` e com a documentação detalhada em `.planning/phases/03-stock-movements-vertical-slice/03-03-SUMMARY.md` §“Idempotency Replay — Two Paths, Same Output”:

1. **Fast-path (chave já existe)** — antes de abrir transação, `StockMovementService.CreateAsync` chama `_movements.GetByIdempotencyKeyAsync(key)`. Se a chave existe, monta a response a partir do movimento existente, marca `IsReplay=true`, devolve `200 OK + Idempotency-Replay: true`. Zero side effect; nenhuma transação aberta.
2. **Slow-path (chave nova)** — service abre transação, faz `SELECT ... FOR UPDATE` no produto, valida invariants (deleted / balance / value-field), tenta INSERT do movimento. Em caminho feliz, UPDATE do produto e COMMIT, retorna `201 Created`.
3. **Race-window recovery (chave nova mas duas requisições simultâneas)** — duas requisições com a mesma `Idempotency-Key` chegam após o fast-path de ambas falhar. Uma INSERT vence; a outra falha com `PostgresException(SqlState='23505')` em `idempotency_key UNIQUE`. O catch faz rollback, re-fetch da chave, monta a response com `IsReplay=true`, devolve `200 OK + Idempotency-Replay: true`. Nenhum produto é atualizado duas vezes.
4. **Garantia: body byte-idêntico** — fast-path e race-recovery produzem **o mesmo body** que a primeira resposta (`201 Created`) exceto pelo status code (200 vs 201) e pelo header `Idempotency-Replay: true`. Verificado via `jq -S` em `.planning/phases/03-stock-movements-vertical-slice/03-VERIFICATION.md` §MOVE-03.